

```
dharanidhar@LAPTOP-2REPP: x + v
dharanidhar@LAPTOP-2REPLIJ:~$ nano week4.c
dharanidhar@LAPTOP-2REPLIJ:~$ cat week4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t child1, child2, child3;
    pid_t zombie;
    int status;

    printf("=====\n");
    printf(" WEEK 4 - PROCESS SYNCHRONIZATION\n");
    printf("=====\n");

    printf("Parent PID: %d\n\n", getpid());

    /* Creating Child 1 */
    child1 = fork();

    if (child1 == 0)
    {
        printf("Child 1 created. PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(1);
    }

    /* Creating Child 2 */
    child2 = fork();

    if (child2 == 0)
    {
        printf("Child 2 created. PID = %d\n", getpid());
        sleep(4);
        printf("Child 2 completed.\n");
        exit(2);
    }
}
```

30°C Partly sunny

Search

ENG IN 15:12 22-08-2026

```

}

/* Creating Child 3 */
child3 = fork();

if (child3 == 0)
{
    printf("Child 3 created. PID = %d\n", getpid());
    sleep(6);
    printf("Child 3 completed.\n");
    exit(3);
}

/* waitpid() demonstration */
printf("\n--- waitpid() Demonstration ---\n");
printf("Parent is waiting specifically for Child 2...\n");

waitpid(child2, &status, 0);

printf("Child 2 collected using waitpid(). PID = %d\n", child2);

/* wait() demonstration */
printf("\n--- wait() Demonstration ---\n");
printf("Parent is waiting for remaining children...\n");

wait(NULL);
wait(NULL);

printf("Remaining children collected using wait().\n");

printf("\n--- Comparison ---\n");
printf("wait() : waits for any child process.\n");
printf("waitpid() : waits for a specific child process.\n");

/* Zombie process demonstration */
printf("\n===== \n");
printf("ZOMBIE PROCESS DEMONSTRATION\n");
printf("===== \n");

zombie = fork();

```

```
zombie = fork();

if (zombie == 0)
{
    printf("Zombie child PID = %d\n", getpid());
    printf("Child is terminating immediately...\n");
    exit(0);
}
else
{
    printf("Parent PID = %d\n", getpid());
    printf("Zombie child PID = %d\n", zombie);

    printf("\nChild has terminated.\n");
    printf("Parent will NOT call wait() immediately.\n");
    printf("Check the process table NOW using:\n");
    printf("ps -el | grep week4\n");

    sleep(15);

    printf("\nParent is now calling waitpid()...\n");

    waitpid(zombie, &status, 0);

    printf("Zombie child collected successfully.\n");
    printf("Zombie process eliminated.\n");
}

printf("\nProgram completed successfully.\n");

return 0;
}

dharanidhar@LAPTOP-2REPPLIJ:~$ gcc week4.c -o week4
dharanidhar@LAPTOP-2REPPLIJ:~$ ./week4
=====
WEEK 4 - PROCESS SYNCHRONIZATION
=====
Parent PID: 1849
```

```

    return 0;
}

dharanidhar@LAPTOP-2REPPLIJ:~$ gcc week4.c -o week4
dharanidhar@LAPTOP-2REPPLIJ:~$ ./week4
=====
WEEK 4 - PROCESS SYNCHRONIZATION
=====
Parent PID: 1849

Child 1 created. PID = 1850
Child 2 created. PID = 1851

--- waitpid() Demonstration ---
Parent is waiting specifically for Child 2...
Child 3 created. PID = 1852
Child 1 completed.
Child 2 completed.
Child 2 collected using waitpid(). PID = 1851

--- wait() Demonstration ---
Parent is waiting for remaining children...
Child 3 completed.
Remaining children collected using wait().

--- Comparison ---
wait() : waits for any child process.
waitpid() : waits for a specific child process.

=====
ZOMBIE PROCESS DEMONSTRATION
=====
Parent PID = 1849
Zombie child PID = 1857

Child has terminated.
Parent will NOT call wait() immediately.
Check the process table NOW using:
ps -el | grep week4
Zombie child PID = 1857
Child is terminating immediately...

```

```
Child 1 created. PID = 1850
Child 2 created. PID = 1851

--- waitpid() Demonstration ---
Parent is waiting specifically for Child 2...
Child 3 created. PID = 1852
Child 1 completed.
Child 2 completed.
Child 2 collected using waitpid(). PID = 1851

--- wait() Demonstration ---
Parent is waiting for remaining children...
Child 3 completed.
Remaining children collected using wait().

--- Comparison ---
wait() : waits for any child process.
waitpid() : waits for a specific child process.

=====
ZOMBIE PROCESS DEMONSTRATION
=====
Parent PID = 1849
Zombie child PID = 1857

Child has terminated.
Parent will NOT call wait() immediately.
Check the process table NOW using:
ps -el | grep week4
Zombie child PID = 1857
Child is terminating immediately...

Parent is now calling waitpid()...
Zombie child collected successfully.
Zombie process eliminated.

Program completed successfully.
dharanidhar@LAPTOP-2REPPLIJ:~$ ps -el | grep week4
dharanidhar@LAPTOP-2REPPLIJ:~$ ps -el | grep week4
dharanidhar@LAPTOP-2REPPLIJ:~$
```